PARSHWANATH CHARITABLE TRUST'S
A.P. SHAH INSTITUTE OF TECHNOLOGY
Department of Computer Science and Engineering
Data Science

Automated Orchestration for SRE and System Administration : AOSS

Group No:13

Yash Baviskar 22107021
Tejas Patil 22107024
Akshata Khandekar 22107034
Mohini Deore 22107065

Project Guide

Dr. Pravin Adivarekar

Contents

- Abstract
- Introduction
- Objectives
- Literature Review
- Research Gap
- Problem Definition
- Scope
- Technological Stack
- Proposed System Architecture/Working/Module Diagram
- Prototype Design Demonstration

Abstract

The **Automated Orchestration Framework for SRE and System Administration (AOSS)** is an AI-powered assistant that executes SRE and admin tasks from natural language instructions. Built on a multi-agent architecture with planning, execution, error-handling, and networking agents, it uses retrieval-augmented generation (RAG) for policy-compliant, context-aware operations across domains like networking, databases, deployments, and file systems.

AOSS offers real-time observability via Grafana dashboards, automated reporting, and self-healing to reduce manual toil. By combining intelligent orchestration with secure, auditable processes, it enhances efficiency, streamlines workflows, and provides a reliable platform for managing complex IT environments.

Sustainable Development Goals(SDG)

- **SDG 8: Decent Work and Economic Growth**
  AOSS reduces manual toil and repetitive tasks in SRE and system administration through safe automation. This enhances efficiency and allows skilled workers to focus on higher-value activities, promoting decent and productive work in the IT sector.
- **SDG 9: Industry, Innovation, and Infrastructure**
  By leveraging AI-driven orchestration with agents and RAG, AOSS improves the reliability, scalability, and maintainability of digital systems. It simplifies complex DevOps workflows, contributing to resilient and modern infrastructure.
- **SDG 11: Sustainable Cities and Communities**
  AOSS supports institutions like colleges and municipal bodies in managing IT systems with greater transparency, reduced errors, and stronger monitoring—enabling safer and more resilient communities.

Introduction

- Despite the advancements in LLM and GenAI capabilities, Most enterprise and organizations do not use AI in Proudction Environment
- Current AI systems are like Crystal Ball with inability to predict output, can be highly unstable causing them be to used in highly sandboxed environment
- Most SRE (Site Reliability Engineering) Task involves toil and grunt work.
- Google has Defined one of the primary objectives in modern SRE to reduce toil (source : Google SRE - What is Toil in SRE: Understanding Its Impact)

Motivations

- “In SRE, we want to spend time on long-term engineering project work instead of operational work. Because the term operational work may be misinterpreted, we use a specific word: toil.”
- SRE Toil : Toil is not just "work I don’t like to do." It’s also not simply equivalent to administrative chores or grungy work. Preferences as to what types of work are satisfying and enjoyable vary from person to person, and some people even enjoy manual, repetitive work. There are also administrative chores that have to get done, but should not be categorized as toil: this is overhead.
- “Our SRE organization has an advertised goal of keeping operational work (i.e., toil) below 50% of each SRE’s time. At least 50% of each SRE’s time should be spent on engineering project work that will either reduce future toil or add service features. Feature development typically focuses on improving reliability, performance, or utilization, which often reduces toil as a second-order effect.”

Objectives

1. **Modular Multi-Agent Orchestration:** Build specialized agents (Planner, Executor, Error-Handler) to execute SRE and sysadmin tasks via natural language across networking, databases, deployments, firewalls, and file systems.
2. **RAG Integration:** Use Chromadb and Neo4j for semantic search and graph-based knowledge linking, grounding agent decisions in documentation, policies, and compliance standards.
3. **Observability & Monitoring:** Log orchestration steps and config changes, with Grafana dashboards tracking uptime, latency, and database metrics for proactive issue detection.
4. **Automated Reporting:** Generate structured daily summaries with JSON logs, PDF/Markdown reports, Slack/email alerts, change diffs, and Prometheus-compatible metrics for auditability.

Literature Review

Sr.no	Title	Author(s)	Year	Methodology	Gaps in Scope
1	Verification and Validation of LLM-RAG for Industrial Automation[1]	Ziran Min, Christof J. Budnik	2025	The paper proposes a Verification and Validation (V&V) framework with a Continuum-Based Failure Classification (CBFC) model that moves beyond binary testing by grading LLM+RAG outputs across retrieval relevance	However, it focuses mainly on evaluation rather than execution, lacks observability and domain-specific compliance checks, introduces overhead in CI/CD pipelines, and is validated only in a narrow industrial use case, limiting applicability to broader sysadmin
2	Revolutionizing Compliance: The Promise of Graph RAG-Based Large Language Models [2]	Kunal Khanvilkar	2025	Proposes a multimodal Graph RAG-based LLM for compliance, constructing knowledge graphs from diverse data sources to enable precise, explainable, and real-time regulatory checks	Implementation is complex, requiring extensive data integration, technical expertise, and ongoing governance to ensure accuracy, transparency, and domain-specific customization.
3	Benchmarking Vector, Graph and Hybrid Retrieval Augmented Generation (RAG) Pipelines for Open Radio Access Networks (ORAN) [3]	Sarat Ahmad, Zeinab Nezami, Maryam Hafeez, Syed Ali Raza Zaidi	2025	The study compares Vector RAG, GraphRAG, and Hybrid GraphRAG pipelines for ORAN by evaluating their performance on question complexity using metrics like faithfulness, answer relevance, context relevance, and factual correctness.	Findings are limited to ORAN specifications and may not generalize to other telecom domains or real-world deployment scenarios.

Sr.no	Title	Author(s)	Year	Methodology	Gaps in Scope
4	AI Observability: Ensuring Trust and Transparency in Machine Learning Pipelines[4]	Rambabu Bandam	2025	Introduces AI observability as a framework for monitoring, analyzing, and visualizing AI models across orchestration, semantic, and model layers, using techniques like LIME and SHAP to enhance explainability, fairness, and compliance.	Faces challenges with complexity, scalability, balancing transparency with performance, and mitigating hallucinations in generative AI.
5	Best Practices Implementing AIOps in Large Organizations[5]	Wallace C. Potts, Christopher Carver	2024	Qualitative case study combining literature review and Veterans Engineering's practical experience to identify best practices in AIOps adoption and interdependency management.	The study is context-specific to federal contractors and may not generalize to all organizations. Its qualitative nature relies on observations and experiences, which may introduce subjectivity and limit reproducibility.
6	Certifying Generative AI: Retrieval-Augmented Generation Chatbots in High-Stakes Environments[6]	Richard Shan	2024	The paper analyzes the use of retrieval-augmented generation (RAG) chatbots in mission-critical settings, focusing on data integrity, response accuracy, and system reliability.	Implementation is complex and may face challenges ensuring reliability, preventing information leakage, and maintaining ethical and safety standards under real-world conditions.

Sr.no	Title	Author(s)	Year	Methodology	Gaps in Scope
7	An Autonomous Agent-based Framework for Self-Healing Power Grid[7]	Zeinab Noorian, Hadi Hosseini, Mihaela Ulieru	2024	A four-layered agent-based system monitors the grid, detects failures, provides temporary recovery, and generates optimized long-term solutions.	The system may face delays in real-time coordination, scalability issues in large interconnected grids, and challenges handling unexpected or complex failures across multiple layers.
8	Agent-Based Software Testing: A Definition and Systematic Mapping Study [8]	Pavithra Perumal Kumaresen, Mirgita Frasheri, Eduard Paul Enoiu	2024	A systematic mapping study reviewing agent-based software testing (ABST) research, analyzing architectures, frameworks, programming languages, and test levels to identify trends and research gaps.	Limited focus on regression testing and potential underrepresentation of non-Java or non-JADE frameworks in current ABST research.
9	Best Practices Implementing AIOps in Large Organizations [9]	Wallace C. Potts, Christopher Carver	2024	Qualitative case study combining literature review and Veterans Engineering's practical experience to identify best practices in AIOps adoption and interdependency management.	The study is context-specific to federal contractors and may not generalize to all organizations. Its qualitative nature relies on observations and experiences, which may introduce subjectivity and limit reproducibility.

Sr.no	Title	Author(s)	Year	Methodology	Gaps in Scope
10	Hybrid Retrieval-Augmented Generation Approach for LLMs Query Response Enhancement [10]	Pouria Omrani, Alireza Hosseini, Kiana Hooshanfar, Zahra Ebrahimian, Ramin Toosi, Mohammad Ali Akhaee	2024 (ICWR Conference, IEEE)	Proposes a hybrid RAG framework combining Sentence-Window and Parent-Child retrieval with a re-ranking mechanism to improve accuracy, relevance, and faithfulness of LLM query responses.	Added complexity and computational overhead may limit scalability and efficiency in large-scale real-time applications.
11	HybridRAG: Integrating Knowledge Graphs and Vector Retrieval Augmented Generation for Efficient Information Extraction [11]	Bhaskarjit Sarmah, Benika Hall, Rohan Rao, Sunil Patel, Stefano Pasquali, Dhagash Mehta	2024	Demonstrates superior retrieval and generation accuracy compared to standalone VectorRAG or GraphRAG, using a novel Q&A dataset from financial earnings call transcripts.	The hybrid approach increases system complexity and may require more computational resources and careful integration of vector and graph retrieval pipelines.
12	Knowledge Graph-Based Retrieval-Augmented Generation for Diabetes-Focused Large Language Models[12]	Maneeha Rani; Bhupesh Kumar Mishra; Dhavalkumar Thakker; Mohammad Nouman Khan	2024	Achieves a Rouge-1 score of 82.19% for diabetes-focused LLMs, providing contextually relevant, evidence-backed responses while addressing hallucinations and source traceability.	The approach requires curated, verified domain-specific knowledge bases and complex integration of multiple retrieval strategies, which may limit scalability to broader healthcare domains.

Sr.no	Title	Author(s)	Year	Methodology	Gaps in Scope
13	To Enhance Graph-Based Retrieval-Augmented Generation (RAG) with Robust Retrieval Techniques [13]	Maneeha Rani, Bhupesh Kumar Mishra, Dhavalkumar Thakker, Mohammad Nouman Khan	2024	Demonstrates a robust retrieval pipeline that mitigates hallucinations, manages unstructured medical literature, and achieves a Rouge-1 score of 82.19% for diabetes-focused LLM responses.	The approach requires carefully curated domain-specific knowledge, preprocessing into knowledge graphs, and integration of multiple retrieval strategies, which may be resource-intensive and limit generalizability to other medical domains.
14	AIOps based Predictive Alerting for System Stability in IT Environment[14]	Pralhad P. Teggi, Harivinod N., and Bharathi Malakreddy	2022	The system collects operational data from OpsBridge and preprocesses it to extract relevant features. Logistic regression is used to predict critical alerts, reducing noise and notifying teams only of genuine issues	The model may miss rare but important failures and struggles to capture complex patterns in IT data. It also has limited scalability and requires retraining for different environments.
15	AI-Governance and Levels of Automation for AIOps-supported System Administration[15]	Anton Gulenko, Alexander Acker, Odej Kao, Feng Liu	2020	Analysis of AIOps automation levels and human–AI shared responsibilities, proposing governance rules for explainable and accountable system administration.	The approach may reduce human control and understanding of automated actions, potentially introducing risks in critical systems. It also relies on predefined rules, which may limit flexibility in dynamic or unforeseen IT environments.

Research Gap (Limitations of existing systems)

- Alert–Action Gap: Current AIOps tools detect and analyze issues but lack safe automated remediation.
- Lack of Guardrails: Generic LLM agents can act but ignore policies, approvals, and safety constraints.
- RAG Limitations: Vector-only misses dependencies; graph-only lacks semantic depth—leading to incomplete context.
- Opaque Automation: Existing observability surfaces infra metrics but not the reasoning behind AI actions, reducing trust.
- Weak Governance: Reporting is manual or inconsistent, leaving compliance and audit gaps.
- These gaps present opportunities that AOSS addresses by introducing state-machine orchestration, hybrid RAG enforcement, structured observability, and automated compliance reporting—positioning it as a safer, accountable path toward closed-loop SRE automation.

Problem Definition

The administration of modern IT infrastructure remains a highly manual, error-prone, and expertise-dependent process. Site Reliability Engineers (SREs) and system administrators are burdened with repetitive tasks, constant context-switching between documentation and consoles, and the pressure of executing complex commands without mistakes. This "toil" leads to operational inefficiencies, burnout, and configuration drift that jeopardizes system security and reliability.

Key Pointers Addressed by AOSS

- **Cognitive Load** → Natural language commands replace complex syntax (e.g., “Deploy payment service with blue-green strategy”).
- **Error Resolution** → Faster MTTR via autonomous failure diagnosis & corrective actions (e.g., port conflicts, dependency issues).
- **Compliance & Safety** → Automated actions aligned with org security policies & standards (PCI-DSS, HIPAA) using version-controlled docs/runbooks.
- **Operational Transparency** → Explainable logs & real-time dashboards ensure trust, moving beyond “black box” automation.
- **Tool Fragmentation** → Unified AI platform to orchestrate across tools (Ansible, Terraform) & domains (networking, DBs, deployments).

Scope

- **AOSS** can be applied in cloud, enterprise, and DevOps environments, enabling automated incident resolution, compliance, and reporting.
- It serves **SREs, DevOps teams, administrators, and compliance officers**, delivering policy-grounded and auditable workflows.
- **Customizable orchestration and knowledge layers** integrate seamlessly with monitoring tools and infrastructure platforms.
- With **state-machine orchestration, hybrid RAG, observability, and reporting**, AOSS ensures safe, transparent, and closed-loop SRE automation.

Technological Stack

Front-end (Visualization & Reporting):

- Grafana 10.4 : Real-time dashboards for observability
- Prometheus 2.52 ,Grafana Loki 2.9 :Metrics & log visualization, alerting
- ReportLab 4.2 / Pandoc 3.1 : Automated PDF/Markdown daily reports

Back-end (Core Orchestration):

- LangChain 0.2 ,LangGraph 0.1 :Multi-agent orchestration using state machines (Planner, Executor, Error-Handler)
- Redis 7.2 :Message broker & task queue for inter-agent communication
- Paramiko 3.4 : Secure SSH-based remote execution on servers
- Docker 25.0 , Docker Compose 2.24 :Containerized deployment with resource isolation

Knowledge & Databases:

- ChromaDB 0.5 : Vector DB for semantic search (runbooks, policies, docs)
- Neo4j 5.18 :Graph DB for dependencies, compliance rules, approvals
- Hybrid RAG Pipeline :Combines semantic retrieval (ChromaDB) with compliance checks (Neo4j)

AI / ML Layer:

- Phi-3 Mini (Quantized, 4-bit) :Lightweight reasoning & planning
- Mistral-7B (Quantized) :Localized LLM for advanced troubleshooting
- Ollama 0.1 :Local model serving & resource management

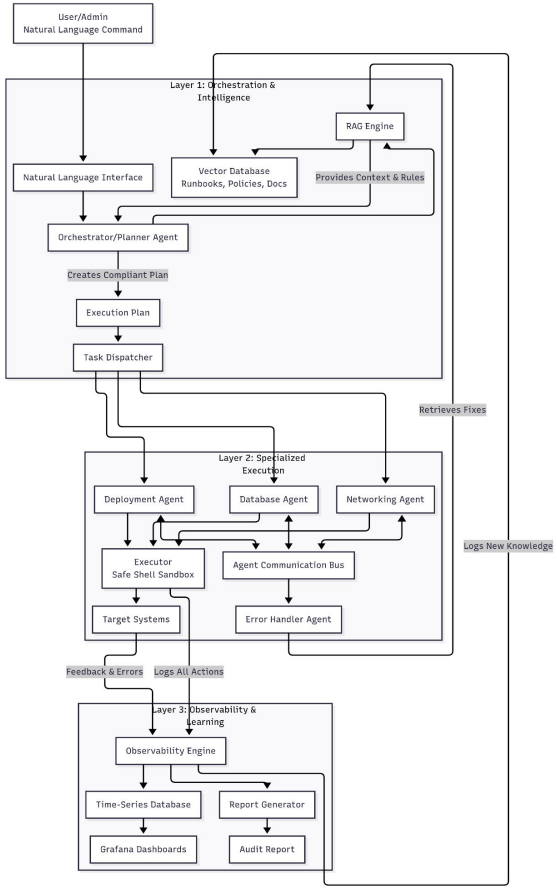
Observability & Governance:

- Structured JSON Logging (Custom Schema) :Traceability with trace_id, agent_role, rag_context_ids
- Prometheus Metrics :Task counters, error rates, latency histograms
- Grafana Dashboards : End-to-end monitoring & compliance view
- Automated Reports (Python + Pandoc/ReportLab) :Daily summaries, error trends, configuration diffs

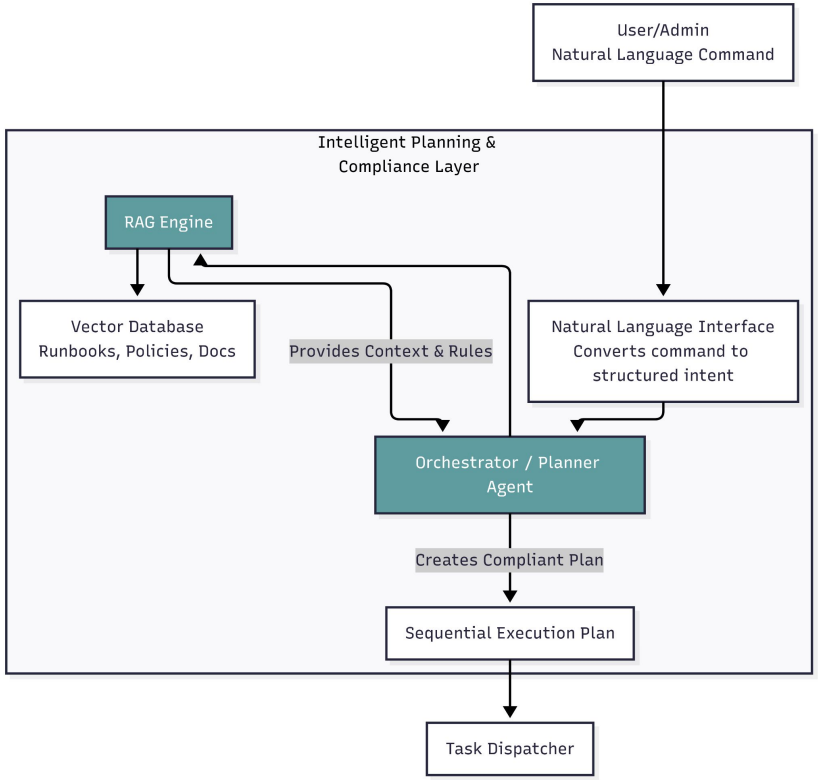
Custom Methodologies (Value Additions):

- State Machine Orchestration : Deterministic & auditable multi-agent execution
- Hybrid RAG Enforcement : Policy-grounded decision-making
- Error-Handler Agent : Context-aware self-healing (beyond retries)
- Resource Optimization : Quantized LLMs ,memory-limited containers (8 GB feasibility)

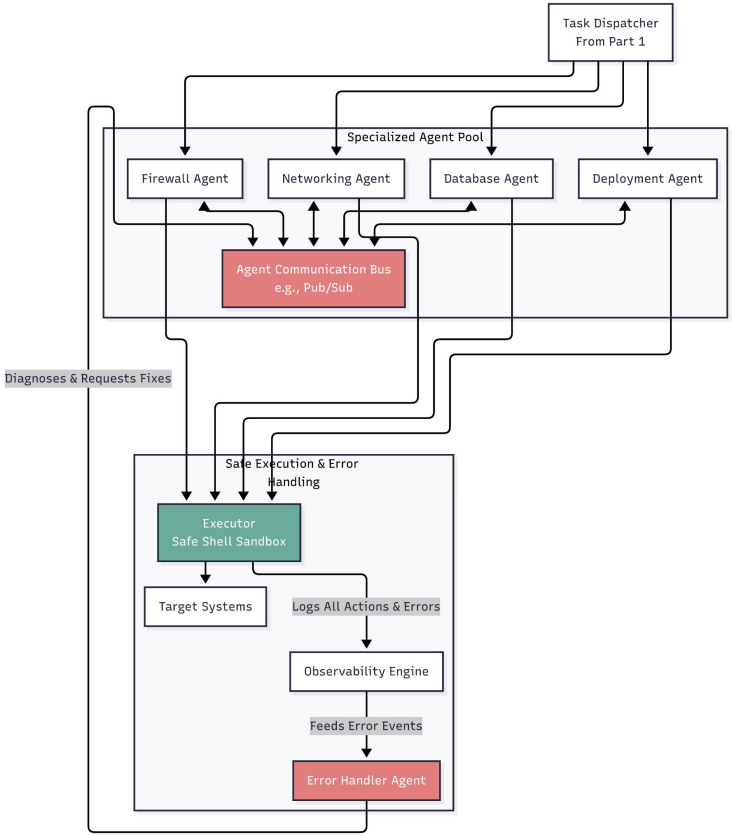
High-Level System Architecture



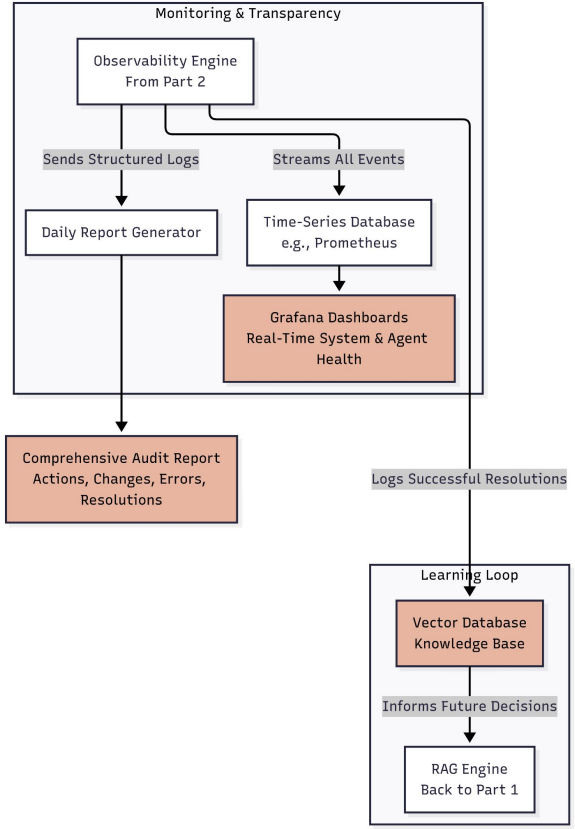
Part 1: User Command & Intelligent Planning



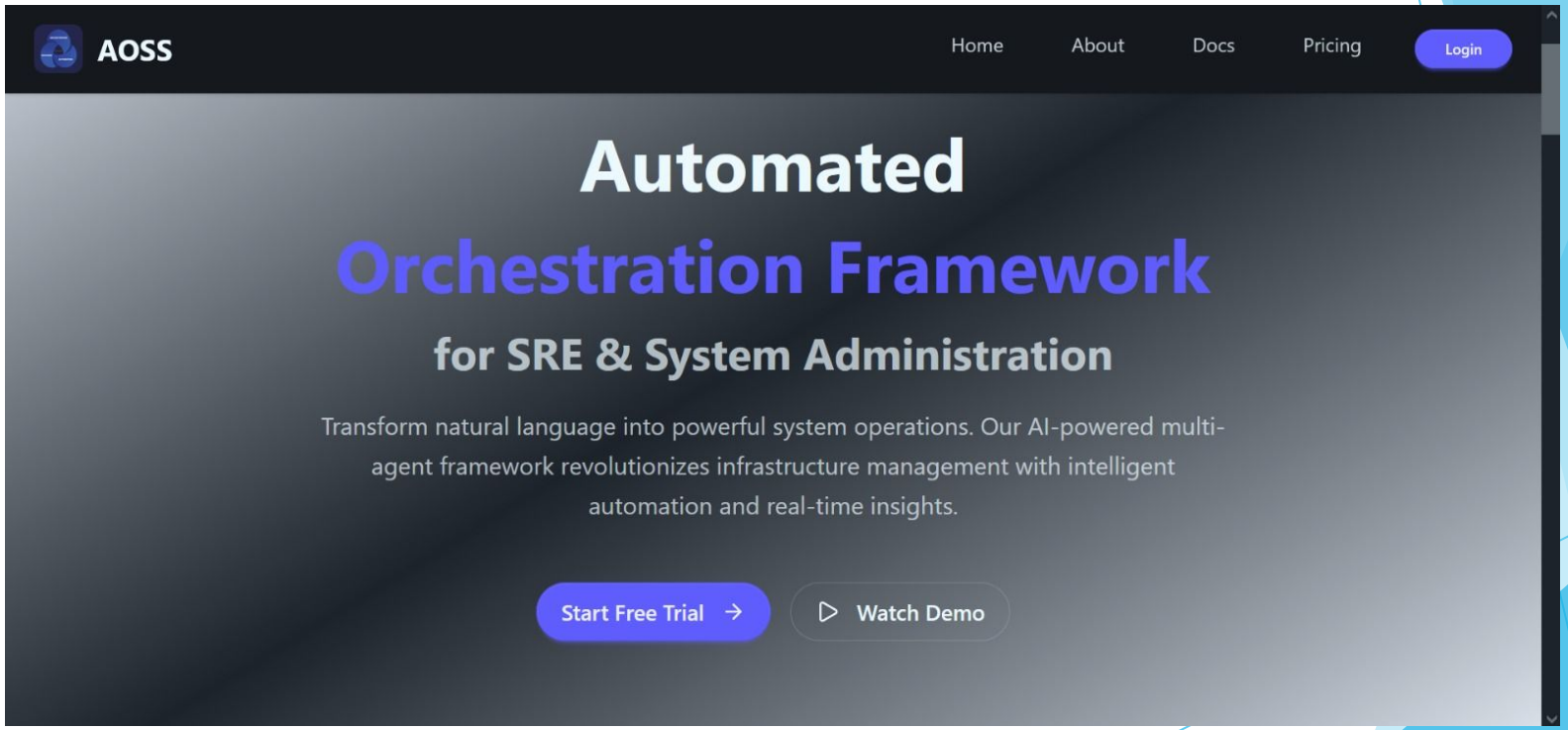
Part 2: Multi-Agent Execution & Self-Healing



Part 3: Observability, Reporting & Learning



Implementation



AOSS

Home About Docs Pricing Login

Powerful Features for Modern Infrastructure

Built for enterprise-scale operations with cutting-edge AI technology

AI-Powered Intelligence

Advanced natural language processing transforms complex requirements into actionable system tasks with intelligent context awareness.

Multi-Agent Architecture

Distributed agent system coordinates seamlessly across infrastructure components, ensuring scalable and resilient operations.

Lightning Fast Execution

Optimized RAG implementation delivers sub-second response times for critical system operations and monitoring tasks.

Enterprise Security

Built-in security protocols and audit trails ensure compliance while maintaining the highest standards of system integrity.

Advanced Analytics

Real-time insights and predictive analysis help prevent issues before they impact your systems and users.

DevOps Integration

Seamlessly integrates with existing CI/CD pipelines and infrastructure as code for unified operations management.

http://localhost:5173/chat

AOSS

Home About Docs Pricing Login

Welcome to AOSS Orchestrator. Enter a natural query to begin automation.

install python

```
→ sudo apt update -y
→ sudo apt install python3 -y
```

```
→ sudo apt update -y
→ sudo apt install python3 -y
```

Execution Series	Status	Server Logging
sudo apt update -y	✅ Success	Hit:1 http://security.ubuntu.com/ubuntu noble-security InRelease Hit:2 http://archive.ubuntu.com/ubuntu noble InRelease Hit:3 http://archive.ubuntu.com/ubuntu noble-updates InRelease Hit:4 http://archive.ubuntu.com/ubuntu noble-backports InRelease Reading package lists... Building dependency tree... Reading state information... 7 packages can be upgraded. Run 'apt list --upgradable' to see them. WARNING: apt does not have a stable CLI interface. Use with caution in scripts.
		Reading package lists... Building dependency tree...

Welcome to AOSS Orchestrator. Enter a natural query to begin automation.

set up nginx and check if it is running

```
→ sudo apt update -y
→ sudo apt install nginx -y
→ sudo systemctl start nginx
→ sudo systemctl enable nginx
→ sudo systemctl status nginx
```

```
→ sudo apt update -y
→ sudo apt install nginx -y
→ sudo systemctl start nginx
→ sudo systemctl enable nginx
→ sudo systemctl status nginx
```

Execution Series	Status	Server Logging
sudo apt update -y	✅ Success	Hit:1 http://security.ubuntu.com/ubuntu noble-security InRelease Hit:2 http://archive.ubuntu.com/ubuntu noble InRelease Hit:3 http://archive.ubuntu.com/ubuntu noble-updates InRelease Hit:4 http://archive.ubuntu.com/ubuntu noble-backports InRelease Reading package lists... Building dependency tree... Reading state information... 7 packages can be upgraded. Run 'apt list --upgradable' to see them. WARNING: apt does not have a stable CLI interface. Use with caution in scripts.
		Reading package lists... Building dependency tree... Reading state information... nginx is already the newest version (1.24.0-2ubuntu7.4). The following packages were automatically installed and are no

sudo systemctl status nginx	✅ Success	```● nginx.service - A high performance web server and a reverse proxy server
		Loaded: loaded (/usr/lib/systemd/system/nginx.service; enabled; preset: enabled)
		Active: active (running) since Mon 2025-08-25 01:41:35 UTC; 6min ago
		Docs: man:nginx(8)
		Main PID: 4530 (nginx)
		Tasks: 9 (limit: 9422)
		Memory: 6.4M (peak: 14.5M)
		CPU: 117ms
		CGroup: /system.slice/nginx.service
		├─4530 "nginx: master process /usr/sbin/nginx -g daemon on; master_process on;"
		├─4532 "nginx: worker process"
		├─4533 "nginx: worker process"
		├─4534 "nginx: worker process"```
		```process"```
		```process"```
		``

References

1. Z. Min and C. J. Budnik, "Verification and Validation of LLM-RAG for Industrial Automation," 2025 IEEE International Conference on Artificial Intelligence Testing (AITest), Tucson, AZ, USA, 2025 [1]
2. K. Khanvilkar, **"Revolutionizing Compliance: The Promise of Graph RAG-Based Large Language Models,"** arXiv:2502.12345, 2025.[2]
3. S. Ahmad, Z. Nezami, M. Hafeez and S. A. R. Zaidi, **"Benchmarking Vector, Graph and Hybrid Retrieval Augmented Generation (RAG) Pipelines for Open Radio Access Networks (ORAN)**," arXiv:2507.03608, 2025.[3]
4. R. Bandam, **"AI Observability: Ensuring Trust and Transparency in Machine Learning Pipelines,"** IEEE Transactions on Artificial Intelligence, Early Access, 2025, doi: 10.1109/TAI.2025.xxxxxxx.[4]
5. W. C. Potts and C. Carver, "Best Practices Implementing AIOps in Large Organizations," in 2024 IEEE International Conference on Smart Applications, Communications and Networking (SmartNets), 2024[5]
6. R. Shan, "Certifying Generative AI: Retrieval-Augmented Generation Chatbots in High-Stakes Environments," IEEE Computer, vol. 57, no. 9, pp. 84–90, Sept. 2024, doi: 10.1109/MC.2024.3401085.[6]
7. Z. Noorian, H. Hosseini and M. Ulieru, "**An Autonomous Agent-based Framework for Self-Healing Power Grid,**" in 2009 IEEE International Conference on Systems, Man and Cybernetics (SMC), pp. 1983–1988, 2009.[7]

6. R. Shan, "Certifying Generative AI: Retrieval-Augmented Generation Chatbots in High-Stakes Environments," IEEE Computer, vol. 57, no. 9, pp. 84–90, Sept. 2024, doi: 10.1109/MC.2024.3401085.[6]
7. Z. Noorian, H. Hosseini and M. Ulieru, "An Autonomous Agent-based Framework for Self-Healing Power Grid," in 2009 IEEE International Conference on Systems, Man and Cybernetics (SMC), pp. 1983–1988, 2009.[7]
8. P. P. Kumaresen, M. Frasheri and E. P. Enoiu, "Agent-Based Software Testing: A Definition and Systematic nt-Based Software Testing: A Definition and Systematic Mapping Study," Information and Software Technology, vol. 173, pp. 107586, 2024, doi: 10.1016/j.infsof.2024.107586.
9. K. Khanvilkar, "Revolutionizing Compliance: The Promise of Graph RAG-Based Large Language Models," arXiv:2502.12345, 2025.
10. R. Bandam, "AI Observability: Ensuring Trust and Transparency in Machine Learning Pipelines," IEEE Transactions on Artificial Intelligence, Early Access, 2025, doi: 10.1109/TAI.2025.xxxxxxx.
11. W. C. Potts and C. Carver, "Best Practices Implementing AIOps in Large Organizations," in 2024 IEEE International Conference on Smart Applications, Communications and Networking (SmartNets), 2024.

12. P. Omrani, A. Hosseini, K. Hooshanfar, Z. Ebrahimian, R. Toosi and M. A. Akhaee, **"Hybrid Retrieval-Augmented Generation Approach for LLMs Query Response Enhancement,"** in 2024 International Conference on Web Research (ICWR), IEEE, 2024.[12]

13. M. Rani, B. K. Mishra, D. Thakker and M. N. Khan, "Knowledge Graph-Based Retrieval-Augmented Generation for Diabetes-Focused Large Language Models," in Proceedings of the IEEE International Conference on Healthcare Informatics (ICHI), 2024.[13]

14. Pralhad P. Teggi, N. Harivinod, and B. Malakreddy, "AIOps based Predictive Alerting for System Stability in IT Environment," in 2022 IEEE International Conference on Innovative Trends in Information Technology (ICITIIT), 2022, doi: 10.1109/ICITIIT54346.2022.9744236.[14]

15. Anton Gulenko, Alexander Acker, Odej Kao, and Feng Liu, "AI-Governance and Levels of Automation for AIOps-supported System Administration," in 2020 IEEE/IFIP Network Operations and Management Symposium (NOMS), 2020, doi: 10.1109/NOMS47738.2020.9110403.[15]

Thank You...!!